UGANDA CHRISTIAN UNIVERSITY

Faculty of Engineering Design and Technology

In-Class Database Activity

Database Normalisation and Integrity Constraints

Scenario: Mukono Health Centre IV

Group Members

No.	Name	Registration Number
1	**AMANYA AARON**	**M26B13/006**
2	**NABUKENYA SARAH**	**M26B13/031**
3	**AMPUMUZA ESTHER**	**M26B13/086**
4	**MIREMBE NORAH**	**M26B13/025**
5	**NAKAFEERO HADIJAH KARAMAGI**	**M26B13/032**
6	**WADEJJE AARON DAVID**	**M26B13/042**
7	**NOKUKUNDAKWE MARTHA**	**M26B13/035**

Course Unit: Database Design and Application

Date of Submission: October 21, 2026

Part A

1. Four pieces of information repeated in the table.

- Patient name (**Amina N.** appears in V101 and V103)
- Patient phone number (**0701001001** appears in V101 and V103)
- Clinician name (**Dr. Namusoke** appears in V102 and V103)
- Medicine name (**Paracetamol** appears in V101, V102 and V103)

2. Two problems if a patient's phone number changes and only one row is updated.

- **Data inconsistency:** the same patient ends up with two different phone numbers, so staff cannot tell which one is correct.
- **Data integrity loss:** reminders, SMS messages may use the old number, so the patient is contacted on the wrong number or missed entirely.

3. One insertion problem and one deletion problem.

- **Insertion problem:** a new clinician or a new medicine cannot be recorded until that clinician or medicine is attached to a visit, so a newly hired doctor or a new drug cannot be entered at all.
- **Deletion problem:** if the only visit for a patient (P021, visit V102) is deleted, all information about that patient is lost.

4. What uniquely identifies each visit? Each patient?

- **Visit:** `VisitNo`: This one is unique per visit.
- **Patient:** `PatientID`: And this is unique per patient.

Part B

5. First Normal Form (1NF).

Rule applied: Every cell must hold a single atomic value; there must be no repeating groups or multi-valued attributes, and each row must be uniquely identifiable.

Is the sample in 1NF? The sample **rows** are atomic (one medicine per row), so the raw sample appears to be 1NF as displayed. However, the design is **not properly normalised** because it mixes several entities (patient, visit, clinician, medicine) in one relation.

Issue with storage: The same visit (V101) is split across two rows just to record a second medicine (ORS), and patient/visit data is duplicated. There is no single clean key: `VisitNo` repeats, so `VisitNo` alone is not unique.

6. Second Normal Form (2NF).

Rule: every non-key attribute must depend on the **whole** composite key.

- `PatientID, PatientName, PatientPhone, ClinicianID, ClinicianName, VisitDate` depend only on `VisitNo` (part of the key).
- `MedicineName` depends only on `MedicineID` (part of the key).

For example: `MedicineName = 'Paracetamol'` depends only on `MedicineID = M08`; `PatientName = 'Amina N.'` depends only on `VisitNo = V101`. Both are partial dependencies, so the table is not in 2NF.

this is how to fix; split into Visit(VisitNo, VisitDate, PatientID, ClinicianID) and Medicine(MedicineID, MedicineName), plus a prescription table (VisitNo, MedicineID, Quantity).

7. Third Normal Form (3NF).

Rule:; non-key attributes must not depend on other non-key attributes.

Example: `ClinicianName` depends on `ClinicianID`, a non-key attribute in the visit table. `ClinicianName` should therefore live in a separate **Clinician** table (ClinicianID, ClinicianName). Similarly, `PatientName`/`PatientPhone` depend on `PatientID`, and `MedicineName` depends on `MedicineID`.

8. Proposed normalised design.

Patient (PK) PatientID

PatientID (PK)	PatientName	PatientPhone
P014	Amina N.	0701001001
P021	Brian K.	0702002002

Clinician (PK) ClinicianID

ClinicianID (PK)	ClinicianName
C03	Dr. Okello
C05	Dr. Namusoke

Medicine (PK) MedicineID

MedicineID (PK)	MedicineName
M08	Paracetamol
M11	ORS

Visit (PK) VisitNo (FK: PatientID, ClinicianID)

VisitNo (PK)	VisitDate	PatientID (FK)	ClinicianID (FK)	Status
V101	03/10/2026	P014	C03	Open
V102	04/10/2026	P021	C05	Open
V103	04/10/2026	P014	C05	Open

Prescription (PK) (VisitNo, MedicineID) (FK: VisitNo, MedicineID)

VisitNo (PK, FK)	MedicineID (PK, FK)	Quantity
V101	M08	12
V101	M11	2
V102	M08	6
V103	M08	4

Foreign keys: Visit.PatientID → Patient.PatientID; Visit.ClinicianID → Clinician.ClinicianID; Prescription.VisitNo → Visit.VisitNo; Prescription.MedicineID → Medicine.MedicineID.

Part C

Business rule	Constraint and table/column
Every patient must have a PatientID, and no two patients can share it.	**PRIMARY KEY** on `Patient.PatientID` (implies NOT NULL and UNIQUE).
A visit must refer to a patient who already exists.	**FOREIGN KEY** on `Visit.PatientID` referencing `Patient.PatientID`.
A patient phone number, when provided, should not be duplicated for two different patients.	**UNIQUE** on `Patient.PatientPhone` (nullable, so "when provided" is respected).
Quantity of medicine issued must be greater than zero.	**CHECK** on `Prescription.Quantity` (`CHECK (Quantity > 0)`).
A patient's name must be entered before saving the patient record.	**NOT NULL** on `Patient.PatientName`.
If a new visit is created without a recorded status, use 'Open' as the starting status.	**DEFAULT** on `Visit.Status` (`DEFAULT 'Open'`).

9. PRIMARY KEY vs FOREIGN KEY.

A **PRIMARY KEY** uniquely identifies each row in its **own** table; it cannot be NULL or duplicated (`Patient.PatientID`). A **FOREIGN KEY** is a column in a **child** table that references the primary key of a **parent** table, enforcing referential integrity (`Visit.PatientID` references `Patient.PatientID`). A foreign key may be NULL (if optional) and may repeat values (many visits per patient).

10. Visit referring to a non-existent PatientID.

Problem: a visit with no matching patient. You cannot tell whose visit it is; patient history, billing and reporting break, and joins silently drop the row.

Constraint that prevents it: the **FOREIGN KEY** on `Visit.PatientID` referencing `Patient.PatientID`, with referential integrity enforced.

Part D

11. Storing Medicine1, Medicine2, Medicine3 in the Visit table.

No, we disagree. This creates a repeating group (a multi-valued attribute spread across columns), which violates 1NF. If a patient receives four medicines, there is no column to store the fourth, you would have to keep adding Medicine4, Medicine5 or the extra medicine is simply lost. Queries such as "total quantity of Paracetamol issued" also become useless, and empty columns waste space. The correct solution is to separate the Prescription table which handles any number of medicines.

12. "If the application checks the data, database constraints are not necessary."

We disagree. and below are our two reasons why:

1. **D**ata can be changed by other applications, scripts, imports or direct SQL, not only the app. Constraints protect integrity regardless of the entry path, and application bugs would otherwise allow bad data.
2. **Constraints are enforced centrally, consistently and declaratively.** They prevent race conditions and concurrent-update anomalies that app-level checks can miss, and they guarantee rules (uniqueness, referential integrity) even for future applications. Relying on the app means every app must re-implement and maintain the same rules.

13. One test to prove a CHECK constraint works.

Test: attempt to insert a prescription with `Quantity = 0` (or -5).

- **Input:** `INSERT INTO Prescription (VisitNo, MedicineID, Quantity) VALUES ('V101','M08', 0);`
- **Expected result:** the insert is **rejected** with a constraint-violation error ("CHECK constraint failed"); no row is added.